NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

ALEXANDRE MIGUEL DA SILVA PERES

BSc in Computer Science and Engineering

PLATFORM FOR DISSERTATION/PROJECT PROPOSAL AND MANAGEMENT

Dissertation Plan
MASTER IN COMPUTER SCIENCE AND ENGINEERING
SPECIALIZATION IN PROGRAMMING LANGUAGES AND SOFTWARE SYSTEMS

NOVA University Lisbon
Draft: *January 17, 2026*

PLATFORM FOR DISSERTATION/PROJECT PROPOSAL AND MANAGEMENT

ALEXANDRE MIGUEL DA SILVA PERES

BSc in Computer Science and Engineering

Adviser: Nuno Preguiça
Full Professor, NOVA University Lisbon

Co-adviser: Vítor Duarte
Assistant Professor, NOVA University Lisbon

Dissertation Plan
MASTER IN COMPUTER SCIENCE AND ENGINEERING
SPECIALIZATION IN PROGRAMMING LANGUAGES AND SOFTWARE SYSTEMS

NOVA University Lisbon
Draft: *January 17, 2026*

Abstract

Regardless of the language in which the dissertation is written, usually there are at least two abstracts: one abstract in the same language as the main text, and another abstract in some other language.

The abstracts' order varies with the school. If your school has specific regulations concerning the abstracts' order, the NOVAthesis LaTeX (novathesis) (LaTeX) template will respect them. Otherwise, the default rule in the novathesis template is to have in first place the abstract in *the same language as main text*, and then the abstract in *the other language*. For example, if the dissertation is written in Portuguese, the abstracts' order will be first Portuguese and then English, followed by the main text in Portuguese. If the dissertation is written in English, the abstracts' order will be first English and then Portuguese, followed by the main text in English. However, this order can be customized by adding one of the following to the file `5_packages.tex`.

```
\ntsetup{abstractorder={<LANG_1>,...,<LANG_N>}}
\ntsetup{abstractorder={<MAIN_LANG>={<LANG_1>,...,<LANG_N>}}}
```

For example, for a main document written in German with abstracts written in German, English and Italian (by this order) use:

```
\ntsetup{abstractorder={de={de,en,it}}}
```

Concerning its contents, the abstracts should not exceed one page and may answer the following questions (it is essential to adapt to the usual practices of your scientific area):

1. What is the problem?
2. Why is this problem interesting/challenging?
3. What is the proposed approach/solution/contribution?
4. What results (implications/consequences) from the solution?

Keywords: One keyword · Another keyword · Yet another keyword · One keyword more · The last keyword

Resumo

Independentemente da língua em que a dissertação está escrita, geralmente esta contém pelo menos dois resumos: um resumo na mesma língua do texto principal e outro resumo numa outra língua.

A ordem dos resumos varia de acordo com a escola. Se a sua escola tiver regulamentos específicos sobre a ordem dos resumos, o template (LaTeX) novathesis irá respeitá-los. Caso contrário, a regra padrão no template novathesis é ter em primeiro lugar o resumo *no mesmo idioma do texto principal* e depois o resumo *no outro idioma*. Por exemplo, se a dissertação for escrita em português, a ordem dos resumos será primeiro o português e depois o inglês, seguido do texto principal em português. Se a dissertação for escrita em inglês, a ordem dos resumos será primeiro em inglês e depois em português, seguida do texto principal em inglês. No entanto, esse pedido pode ser personalizado adicionando um dos seguintes ao arquivo `5_packages.tex`.

```
\abstractorder(<MAIN_LANG>):={<LANG_1>,...,<LANG_N>}
```

Por exemplo, para um documento escrito em Alemão com resumos em Alemão, Inglês e Italiano (por esta ordem), pode usar-se:

```
\ntsetup{abstractorder={de={de,en,it}}}
```

Relativamente ao seu conteúdo, os resumos não devem ultrapassar uma página e frequentemente tentam responder às seguintes questões (é imprescindível a adaptação às práticas habituais da sua área científica):

1. Qual é o problema?
2. Porque é que é um problema interessante/desafiante?
3. Qual é a proposta de abordagem/solução?
4. Quais são as consequências/resultados da solução proposta?

Palavras-chave: Primeira palavra-chave · Outra palavra-chave · Mais uma palavra-chave · A última palavra-chave

Contents

List of Figures

List of Tables

Acronyms

AI Artificial Intelligence *(pp. 4, 5)*
API application programming interface *(pp. 5–7)*

BaaS Backend-as-a-Service *(pp. v, 4–6)*
BPMN Business Process Model and Notation *(p. 3)*

CRUD Create, Read, Update, Delete *(p. 6)*

DI Department of Computer Science *(pp. 1, 3, 5)*

EJS Embedded JavaScript *(p. 7)*
ERD Entitiy-Relationships Diagrams *(p. 3)*

FCT NOVA School of Science and Technology *(pp. 1, 3, 5)*

HTML HyperText Markup Language *(p. 7)*

I/O Input/Output *(p. 7)*

JPA Java Persitence API *(p. 6)*

LLM large language models *(p. 5)*

NoSQL Not only SQL *(p. 4)*
NOVA NOVA University Lisbon *(pp. 1, 3, 5)*
novathesis NOVAthesis LaTeX *(pp. i, iii)*

RLS Row-Level-Security *(p. 6)*

SQL Structured Query Language *(pp. 4, 6)*

UX User Experience *(pp. 2, 4)*

1

Introduction

1.1 Context and Background

The transition of students from academic life to the professional market or advanced research is a centerpiece of the university experience, essentially enabled through three diferent distinct paths at the Department of Computer Science (DI) of NOVA School of Science and Technology (FCT):

- Undergraduate Internships: Primarly offered by external companies to provide students with their first professional experience.
- Master's Dissertations: Scientific thesis typically conducted within the university, focusing on academic research under the supervision of professors.
- Engineering Projects: Practical projects proposed by external companies or the Department of Informatics itself, allowing Master's students to apply advanced engineering priciples to real-world projects.

Managing those processes involves a complex coordination of multiple stakeholders: students seeking opportunities aligned with their skills and interests, professors overseeing academic quality, and external companies providing practical challenges. Within this ecosystem, a reliable digital platform is essential to act as a central hub for communication, proposal submission, and student placement.

1.2 Problem Statement

At DI @NOVA FCT, the management of internships, dissertations and engineering projects involves a multi-stage lifecycle that inculdes proposal submission, clarification and approval processes, making proposals available to students, report/dissertation submission, and providing documentation to juries. Currently the department relies on a legacy platform to support some of these activities, however this system presents several critical issues that hinder the efficienty of the department.

The most significant limitation is that the current platform is not being utilized for the Master's degree processes - including both scientific dissertations and engineering projects. At the moment it is exclusively used to manage undergraduate internships, meaning that the more complex workflows associated with Master's students are handled through manual, or non-standardized methods. This creates a lack of centralized data an increases the administrative burden on coordinators, professors and the secretatriat.
Furthermore, even within its current scope, the legacy system suffers from several functional and technological gaps:

- Communication Gaps and Reliability: The current method for "Call for Proposals" is manual, requiring coordinators to send individual emails to companies, advertisting the opportunity send proposals, for each edition. There are no safeguards to ensure that these emails follow specific deliverability rules, often resulting in messages being flagged as spam. Additionally, the system lacks the flexibility to create and edit email templates, preventing coordinators from customizing communication for different editions or context.

- Inneficient Feedback Loops: Although companies can submit proposals, there is no integrated mechanism for coordinators to provide direct feedback on those proposals for clarification. This forces corrections to be handled outside the platform, whereas a centralized system would allow companies to correct and resbumit proposals based on coordinator comments.

- Stale Company Data: The existing repository contains a large amout of outdated information, including companies that no longer exist or have changed their contacts or other information. There is a lack of a "validation" status to ensure only active and verified companies can send proposals.

- Manual Protocol Management: For new companies that have never participated in previous editions, the process of signing the required protocols is still enteriely manual. The platform should automate delivery and tracking of these legal documents as soon as a new comapny registers.

- Outdated User Experience (UX): The interface is not intuitive for external stakeholders, making it difficult for comapnies to register, manage their employees accounts, or monitor the status of their proposals in real-time.

In conclusion, the necessity for a new platform arises from the need to unify all academic degrees under a single digital infrastrcture, replacing manual tracking with a secure, automated system that ensures data integrity and seamless experience for all stakeholders.

1.3 Objectives

The primary objective of this dissertation is to design and develop a modernized web-based platform that unifies and automates the management and entire lifecycle of undergraduate internships, Master's dissertations, and engineering projects for DI@NOVA FCT. The projects aims to transition from a fragmented, manual system to a centralized digital ecosystem.
To achieve this goal, the following specific objectives have been defined for the initial version:

- Process Unification: Integrate the lifecycles of undergraduate internships and Master's degree pathways (both scientific and engineering-focused) into a single platform, replacing the current manual methods.
- Workflow Automation: Implement automated procedures for the "Call for Proposals student seriation (ranking), and the validation of proposals.
- Enhanced Communication and Feedback: Establish direct feedback loops within the platform, allowing coordinators to request clarifications on proposals and ensuring companies can resubmit corrected versions.
- Document and Protocol Management: Automate the delivery, tracking, and storage of legal protocols, student CVs, and recommendation letters.
- Administrative Oversight: Develop advanced administrative tools, including "superuser" access for student support, detailed system logs for auditing, and validation workflow for new company registrations.

1.4 Methodology

This project will follow an Agile/Scrum methodology, characterized by iterative development, frequent feedback loops [7] to ensure the final platform aligns with the university's complex requirements. The work plan is structured into several phases, prioritizing rapid prototyping followed by a transition to a high-performance custom architecture.
To achieve this goal, the following specific objectives have been defined for the initial version:

1. Requirement Gathering and Analysis: A through analysis of the legacy system and current manual processes was conducted to identify functional and non-functional requirements. This phase includes the definition of user roles (Admin, Coordinator, Student, Professor, Company, Secretariat) and their respective use cases.
2. System Modeling and Design: Utilizing Business Process Model and Notation (BPMN) to map complex workflows [5] and Entitiy-Relationships Diagrams (ERD)

to design the database schema [2]. This phase also involves planning the system architecture.

3. AI-Assisted Prototyping (Vibe Coding): The initial implementation phase will leverage Vibe Coding tools, such as Lovable, Figma Make or Google AI Studio, to rapidly generate the frontend and core functional interfaces [3]. These tools excel at creating React-based user interfaces through iterative natural language prompting. During this stage, Supabase will be utilized as Backend-as-a-Service (BaaS) to provide an immediate infrastructure for authentication, database management, and initial data structures via SQL[9]. This approach allows for the rapid fulfillment of most functional requirements while maintaining a live, testable version of the platform.

4. Architectural Transition and Manual Development: Once the prototype reaches a stable state and meets the primary functional requirements, the project will transition from AI-generated code to manual developlment. This move is essential to refine the User Experience, optimize performance, and implement custom features that go beyond the limitations of AI-driven boilerplate.

5. Custom Backend Implementation: To support the department's more complex business logic - such as the specialized student seriation (ranking) algorithms and robust administrative auditing - the system will transition to a dedicated custom backend, likely using Spring Boot [8]. While Supabase is effective for rapid prototyping, a Spring Boot architecture offers superior domain modeling, type safety and maintanability through its layered structure (Controller, Service, Repository). This transition ensures that the platform is not only fast to develo intially but also scalable and reliable for long-term university use.

6. Verification and Validation: The platform will be tested against the initial requirements, ensuring that automated emails follow specific deliverability rules avoiding being flagged as spam.Additionally, the seriation logic will be rigourously validated to ensure it correctly handles student placements and matching according to pre-defined criteria.

7. Documentation and Evaluation: There will be a continuous documentation of the architecture decisions, providing detailed justications for technical choices such as the preference for SQL over NoSQL for structured academic data, and other technical choices.

2

State of the art and Technologies

This chapter examines the current state of software engineering practices and technologies relevant to building a modern platform for the DI at NOVA FCT. The goal is to overcome limitations of the legacy system - such as fragmented data, manual workflows, and inefficient comunication - by systematically evaluating contemporary approaches to backend architecutres, data persitence, frontend frameworks, and AI-assisted devleopment workflows. Particular attention is given to the emerging paradigm of AI-assisted software development, often described as "Vibe Coding" [3] which supports rapid prototyping by allowing developers to steer large language models (LLM) through natural-language interactions instead of traditional line-by-line coding.

Modern web application stacks are chacterized by a wide range of choices at each layer, from infrastructure and backend services to frontend frameworks and development workflows. In the backend, developers must choose between fully managed BaaS platforms [4] and custom backends that offer fine-grained control over domain logic, performance, and governance . On the frontend, single-page applications and component-based frameworks (such as React, Angular, or Vue) are commonly combined with RESTful or GraphQL APIs, enabling decoupled client-server architectures and iterative delivery of new features.

Beyond classical architecutres, AI-assisted developlment tools and practices are reshaping how software is desiged and implemented. Under the "Vibe Coding" paradigm, developers describe desired functionality to an LLM, which generates and refines code iteratively, prioritizng rapid expermientation and high-level intent over manual code authoring. This approach align with agile or small teams to assemble complex prototypes quickly, though it raises questions about maintainability, security, and auditability in production systems. [6]

2.1 Backend Architectures and BaaS

The choice of backend architecutre directly influences development speed, scalability, operational complexity, and the ability to encode complex business rules such as student

seriation and multi-role access control. For this project, two main paradigms are considered: Backend-as-a-Service solutions, which offer ready-made infrastructure and managed services, and custom backends built with mature frameworks such as Spring Boot and Express.js, which provide full control over domain modeling, integration patterns, and deployment environments. [1]

BaaS platforms typically optimize for rapid time-to-market by abstracting away server provisioning, database management, and authentication, making them well-suited for early-stage prototyping and small teams. In contrast, custom backends allow tailoring of data models, ownerships of the entire codebase, and fine-grained observability, which becomes critical for institutional applications that must satisfy compliance, long-term maintainability, and integration with existing academic processes.

2.1.1 Supabase (Backend-as-a-Service)

Supabase is an open-souce BaaS stack that builds on PostgreSQL and exposes automatically generated REST APIs through PostgREST, significantly reducing boilerplate CRUD implementation effort. By providing an integrated suite of tools - including database, authentication, storage, and real-time capabilities - Supabase enables rapid development cycles while leveraging a relational data model that is compatible with established SQL tooling and practices.

A central feature of Supabase is its built-in authentication system, which supports user registration, password-based login, magic links, and role-based access control, simplifying the enforcement of different permission profiles for students, companies, professors and administrators. Row-Level-Security (RLS) policies can be defined directly at the database level, ensuring that authorization rules are enforced close to the data and cannot be bypassed by misconfigured API clients. For logic that goes beyond standard CRUD operations - such as orchestrating notification workflows or managing student seriation - Supabase offers serverless "Edge Functions", typically written in TypeScript or JavaScript, which run in a managed environment near end users to minimize latency.

2.1.2 Custom Backend: Spring Boot

Spring Boot is a widely adopted framework for building production-grade Java or Kotlin backends and is especially suitable for complex, domain-driven systems that require strict typing, layered architectures, and extensive integration capabilities. Its opinionated configuration model and extensive ecosystem (Spring Data, Spring Security, Spring Cloud) allow developers to structure applications into controllers, services, and repositories, facilitating clear separation of concerns and maintanable domain models.

For academic workflows that must support student ranking algorithms, complex eligibility rules, and auditable decision-making processes, the type safety and testability provided by Spring Boot are particularly advantageous. The framework integrates with JPA and tools like Hibernate Envers to support automatic auditing of entity changes,

which helps track user actions, state transitions, and login-related events - capabilities that are often required for administrative oversight and internal compliance in university environments.

2.1.3 Custom Backend: Express.js (Node.js)

Express.js is a minimalist web framework for Node.js that offers a lightweight and flexible foundation for building HTTP APIs using JavaScript or TypeScript. Its event-driven, non-blocking I/O model allows a single process to handle many concurrent connections efficiently, making it well-suited for notification-heavy workloads such as broadcasting "Call for Proposals" or status updates to large groups of students.

The Node.js ecosystem provides a rich set of libraries for templated email generation and delivery, including tools such as Handlebars or EJS to render dynamic HTML email bodies for coordinators and administrators. Combined with tasks schedulers or message queues, Express-based backends can orchestrate asynchronous communications and background jobs, helping address a key functional gap of legacy systems that rely on manual email workflows and ad hoc templates.

2.2 Data Persitence: SQL vs. NoSQL

2.3 Frontend Frameworks and User Experience

2.4 AI-Assisted Development (Vibe Coding)

3

Requirements and Current System Analysis

3.1 Analysis of the Legacy System

3.2 Requirementes Gathering

3.3 Use Cases and Stakeholders

4

Solution Proposal

4.1 Proposed System Architecture

4.2 Process Modelling (BPMN)

4.3 Data Model (ERD)

4.4 API and Component Design

4.5 Work Plan

5

A Short LaTeX Tutorial with Examples

This Chapter aims at exemplifying how to do common stuff with LaTeX. We also show some stuff which is not that common! ;)

Please, use these examples as a starting point, but you should always consider using the *Big Oracle* (aka, Google, your best friend) to search for additional information or al-ternative ways for achieving similar results.

5.1 Document Structure

5.2 Dealing with Bibliography

Citing something online [**wiki:shuntingyard**, **flex**, **bison**].

5.3 Inserting Tables

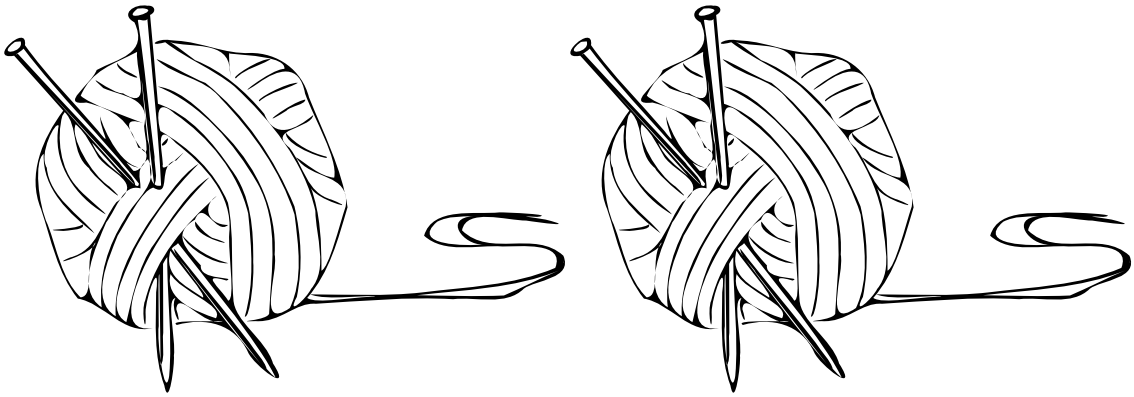
5.4 Importing Images

5.5 Floats, Figures and Captions

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.



(a) One sub-figure (b) Another sub-figure

Figure 5.1: A figure with two sub-figures!

And this is a small text that references the Figure 5.1 and its Subfigures 5.1(a) and 5.1(b).

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan

eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

5.6 Text Formatting

5.7 Generating PDFs from LaTeX

5.7.1 Generating PDFs with pdflatex

You may create PDF files either by using `latex` to generate a DVI file, and then use one of the many DVI-2-PDF converters, such as `dvipdfm`.

Alternatively, you may use `pdflatex`, which will immediately generate a PDF with no intermediate DVI or PS files. In some systems, such as Apple, PDF is already the default format for LaTeX. I strongly recommend you to use this approach, unless you have a very good argument to go for `latex` + `dvipdfm`.

A typical pass for a document with figures, cross-references and a bibliography would be:

```
$ pdflatex template
$ bibtex template
$ pdflatex template
$ pdflatex template
```

You will notice that there is a new PDF file in the working directory called `template.pdf`. Simple :)

Please note that, to be sure all table of contents, cross-references and bibliographic citations are up-to-date, you must run `latex` once, then `bibtex`, and then `latex` twice.

5.7.2 Dealing with Images

You may process the same source files with both `latex` or `pdflatex`. But, if your text include images, you must be careful. `latex` and `pdflatex` accept images in different (exclusive) formats. For `latex` you may use EPS ou PS figures. For `pdflatex` you may use JPG, PNG or PDF figures. I strongly recommend you to use PDF figures in vectorial format (do not use bitmap images unless you have no other choice).

5.7.3 Dealing with Citations

Para fazer citações, deverá usar-se a chave da referência no ficheiro BibTeX. Se for uma única referência [**Artho04**], usar um "~" para ligar o `\cite{...}` à palavra que o precede (...`referência~\cite{Artho04}`). Caso queira fazer múltiplas citações [**Shavit95**, **Silberschatz06**, **Moss85**], deverá agrupá-las dentro de um úinico `\cite{...}`.

Note que o ficheiro de bibliografia pode ter tantas entradas quantas quiser. Apenas aquelas cuja chave seja referenciada no texto é que serão incluidas na listagem de bibliografia.

5.7.4 Footnotes

Footnotes[1] will be numbered and shown in the bottom of the page.

5.7.5 Tables

The Table 5.1 illustrates some important concepts associated with table construction:

i) Do not use vertical lines;

ii) The caption should be above the table;

iii) Use the macros `\toprule`, `\midrule` and `\bottomrule` to make the top, inner and bottom horizontal lines, respectively.

Table 5.1: Test results summary.

Test	Anomalies	Warnings	Correct	Categories	Missed
Connection [**Beckman08**]	2	2	1	*C*	1
Coordinates'03 [**Artho0**	1	4	1	*2B, 1C*	0
Local Variable [**Artho03**]	1	2	1	*A*	0
NASA [**Artho03**]	1	1	1	—	0
Coordinates'04 [**Artho04**]	1	4	1	*3C*	0
Buffer [**Artho04**]	0	7	0	*2A, 1B, 2C, 2D*	0
Double-Check [**Artho04**]	0	2	0	*1A, 1B*	0

[1] This is a simple footnote.

StringBuffer [**Flanagan0**	1	0	0	—	1
Account [**Praun03**]	1	1	1	—	0
Jigsaw [**Praun03**]	1	2	1	*C*	0
Over-reporting [**Praun03**]	0	2	0	*1A, 1C*	0
Under-reporting [**Praun03**]	1	1	1	—	0
Allocate Vector [**IBM-Rep**]	1	2	1	*C*	0
Knight Moves [**Beckman08**]	1	3	1	*2B*	0
Total	12	33	10	5A, 6B, 10C, 2D	2

5.7.6 Figures

The images inserted in the document must be of good quality, preferably in vector format (vector PDF) and not in *bitmap* (PNG, JPG, etc.). *bitmap* images (Figure 5.2) do not scale well and have negative effects on the quality of your document. On the other hand, *vector* images Figure 5.3 scale as much as necessary without degrading the quality of the image.

You should only use *screenshots* for your plots, charts, etc, if you absolutely have no other alternative. Instead of generating a *screenshot*, try using a virtual PDF printer and printing to a PDF file. As a general rule, you will get a vector PDF. Even if your PDF contains images, they will always be of higher or equal quality than what you would get with a *screenshot*.

To combine several figures into a single one... You can then reference the set as Figure 5.4 or the sub-figures separately as 5.4(a) and 5.4(b).

5.8 Equations

LaTeX is a powerful tool for writing in a mathematical style. It allows you to insert formulas into the text, such as this: $ax^2 + bx + c = 0$. It also allows formulas to be highlighted on a separate line and centered on the page.

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

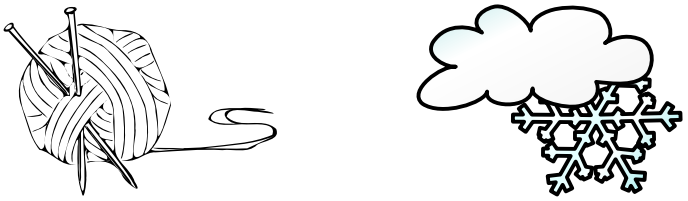
or numbererd

$$e = mc^2 \tag{5.1}$$

which can latter be referenced as equation 5.1

Figure 5.2: Bitmap image (JPG/PNG)

Figure 5.3: Vectorial image (PDF)



(a) Novelo de lã (b) Tempestade com neve

Figure 5.4: Exemplo de utilização de *subbottom*

5.9 Test for algorithms

Uncomment the algorithms source below and add the following to file "`5_packages.tex`"

```
\usepackage{algorithm2e}
\RestyleAlgo{ruled}
```

and uncomment

`\ntaddlistof{listofalgorithms}`

in file "`8_list_og.tex`".

Bibliography

[1] B. Biskupski and A. Pytko-Włodarczyk. *Backend as a Service (BaaS) vs. custom backend: Which one to choose and why?* Accessed: 2026-01-17. 2019. URL: https://www.merixstudio.com/blog/backend-service-baas-vs-custom-backend (cit. on p. 6).

[2] P. P. Chen. "The Entity-Relationship Model: Toward a Unified View of Data". In: *ACM Transactions on Database Systems* 1.1 (1976), pp. 9–36 (cit. on p. 4).

[3] W. contributors. *Vibe coding - chat based AI-assisted software development definition*. Accessed: 2026-01-17. 2026. URL: https://en.wikipedia.org/wiki/Vibe_coding (cit. on pp. 4, 5).

[4] O. Glib. *Backend as a Service use Cases: How to apply*. Accessed: 2026-01-17. 2024. URL: https://acropolium.com/blog/baas-use-cases/ (cit. on p. 5).

[5] Object Management Group. *Business Process Model And Notation (BPMN) Version 2.0*. OMG Document Number: formal/2011-01-03. Version 2.0. Object Management Group, 2011 (cit. on p. 3).

[6] J. Schibelli. *Vibe Coding: How AI is Transforming Software Development*. Accessed: 2026-01-17. 2025. URL: https://dev.to/johnschibelli/the-emergence-of-vibe-coding-how-ai-is-revolutionizing-software-development-4275 (cit. on p. 5).

[7] K. Schwaber and J. Sutherland. *The 2020 Scrum Guide*. Accessed: 2026-01-17. 2020. URL: https://scrumguides.org/ (cit. on p. 3).

[8] Spring Team. *Spring Boot Reference Documentation*. 3.x. Version 3.x. VMware, Inc. 2026. URL: https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/ (cit. on p. 4).

[9] Supabase. *Supabase: The Open Source Firebase Alternative*. https://github.com/supabase/supabase. Project README. 2019 (cit. on p. 4).